

Chapter 1. About the SDK

1.1. Introduction

The SDK (Software Development Kit) provides the headers, libraries and build system necessary to write programs for RP-series microcontroller-based devices such as Raspberry Pi Pico or Raspberry Pi Pico 2 in C, C++ or Arm assembly language.

The SDK is designed to provide an API and programming environment that is familiar both to non-embedded C developers and embedded C developers alike. A single program runs on the device at a time with a conventional `main()` method. Standard C/C++ libraries are supported along with APIs for accessing the RP-series microcontroller's hardware, including DMA, IRQs, and the wide variety fixed function peripherals and PIO (Programmable IO).

Additionally, the SDK provides higher level libraries for dealing with timers, synchronization, Wi-Fi and Bluetooth networking, USB and multicore programming. These libraries should be comprehensive enough that your application code rarely, if at all, needs to access hardware registers directly. However, if you do need or prefer to access the raw hardware registers, you will also find complete and fully-commented register definition headers in the SDK. There's no need to look up addresses in the datasheet.

The SDK can be used to build anything from simple applications, fully-fledged runtime environments such as MicroPython, to low level software such as the RP-series microcontroller's on-chip bootrom itself.

The design goal for entire SDK is to be simple but powerful.

Looking to get started?

This book documents the SDK APIs, explains the internals and overall design of the SDK, and explores some deeper topics like using the PIO assembler to build new interfaces to external hardware. For a quick start with setting up the SDK and writing SDK programs, [Getting started with Raspberry Pi Pico-series](#) is the best place to start.

1.2. Anatomy of a SDK Application

Before going completely depth-first in our traversal of the SDK, it's worth getting a little breadth by looking at one of the SDK examples covered in [Getting started with Raspberry Pi Pico-series](#), in more detail.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/blink_simple/blink_simple.c

```

1 /**
2  * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 #include "pico/stdlib.h"
8
9 #ifndef LED_DELAY_MS
10 #define LED_DELAY_MS 250
11 #endif
12
13 #ifndef PICO_DEFAULT_LED_PIN
14 #warning blink_simple example requires a board with a regular LED
15 #endif
16

```

```

17 // Initialize the GPIO for the LED
18 void pico_led_init(void) {
19     #ifdef PICO_DEFAULT_LED_PIN
20         // A device like Pico that uses a GPIO for the LED will define PICO_DEFAULT_LED_PIN
21         // so we can use normal GPIO functionality to turn the led on and off
22         gpio_init(PICO_DEFAULT_LED_PIN);
23         gpio_set_dir(PICO_DEFAULT_LED_PIN, GPIO_OUT);
24     #endif
25 }
26
27 // Turn the LED on or off
28 void pico_set_led(bool led_on) {
29     #if defined(PICO_DEFAULT_LED_PIN)
30         // Just set the GPIO on or off
31         gpio_put(PICO_DEFAULT_LED_PIN, led_on);
32     #endif
33 }
34
35 int main() {
36     pico_led_init();
37     while (true) {
38         pico_set_led(true);
39         sleep_ms(LED_DELAY_MS);
40         pico_set_led(false);
41         sleep_ms(LED_DELAY_MS);
42     }
43 }

```

This program consists only of a single C file, with three functions. As with almost any C programming environment, the function called `main()` is special, and is the point where the language runtime first hands over control to your program. In the SDK the `main()` function does not take any arguments. It's quite common for the `main()` function not to return, as is shown here.

NOTE

The return code of `main()` is ignored by the SDK runtime, and the default behaviour is to hang the processor on exit.

At the top of the C file, we include a header called `pico/stdlib.h`. This is an umbrella header that pulls in some other commonly used headers. In particular, the ones needed here are `hardware/gpio.h`, which is used for accessing the general purpose I/Os on RP-series microcontrollers (the `gpio_xxx` functions here), and `pico/time.h` which contains, among other things, the `sleep_ms` function. Broadly speaking, a library whose name starts with `pico` provides high level APIs and concepts, or aggregates smaller interfaces; a name beginning with `hardware` indicates a thinner abstraction between your code and the RP-series microcontroller on-chip hardware.

So, using mainly the `hardware_gpio` and `pico_time` libraries, this C program will blink an LED connected to the default LED GPIO (which exact pin varies from one RP-series microcontroller board to another) on and off, twice per second, forever (or at least until unplugged). In the directory containing the C file (you can click the link above the source listing to go there), there is one other file which lives alongside it.

Directory listing of `pico-examples/blink_simple`

```

blink_simple
├── blink_simple.c
└── CMakeLists.txt

0 directories, 2 files

```

The second file is a `CMake` file, which tells the SDK how to turn the C file into a binary application for an RP-series microcontroller-based board. Later sections will detail exactly what `CMake` is, and why it is used, but we can look at the

contents of this file without getting mired in those details.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/blink_simple/CMakeLists.txt

```

1 add_executable(blink_simple
2     blink_simple.c
3 )
4
5 # pull in common dependencies
6 target_link_libraries(blink_simple pico_stdlib)
7
8 # create map/bin/hex/uf2 file etc.
9 pico_add_extra_outputs(blink_simple)
10
11 # call pico_set_program_url to set path to example on github, so users can find the source
    for an example via picotool
12 example_auto_set_url(blink_simple)

```

The standard CMake function `add_executable` in this file declares that a program called `blink_simple` should be built from the C file shown earlier. This is also the "target" name in CMake, and is also used when building the program individually. For example, in the `pico-examples` repository you can say `make blink_simple` in your build directory, and that name comes from *this* line. You can have multiple executables in a single project, and the `pico-examples` repository is one such project.

The `target_link_libraries` is pulling in the SDK functionality that our program needs. If you don't ask for a library, it doesn't appear in your program binary. Just like `pico/stdlib.h` is an umbrella header that includes things like `pico/time.h` and `hardware/gpio.h`, `pico_stdlib` is an umbrella *library* that makes libraries like `pico_time` and `hardware_gpio` available to your build, so that those headers can be included in the first place, and the extra C source files are compiled and linked. If you need less common functionality, not included with `pico_stdlib`, like accessing the DMA hardware, you should add those dependencies here (e.g. listing `hardware_dma` before or after `pico_stdlib`).

We could end the CMake file here, and that would be enough to build the `blink_simple` program. By default, the build will produce an ELF file (executable linkable format), containing all of your code and the SDK libraries it uses. You can load an ELF into the RP-series microcontroller's RAM or external flash through the Serial Wire Debug port, with a debugger setup like `gdb` and `openocd`, or via `picotool`. It's often easier to program your Pico-series device or other RP-series microcontroller board directly over USB with BOOTSEL mode, and this requires a different type of file, called UF2, which serves the same purpose here as an ELF file, but is constructed to survive the rigours of USB mass storage transfer more easily. The `pico_add_extra_outputs` function declares that you want a UF2 file to be created, as well as some useful extra build output like disassembly and map files.

i NOTE

The ELF file is converted to a UF2 using `picotool`.

The final `example_auto_set_url` function is used to embed a link back to the example soource code on github into the output binary such that it can be displayed via `picotool info blink_simple.elf`. You'll see this on the `pico-examples` applications, but it's not applicable to your own programs.

Finally, a brief note on the `pico_stdlib` library. Besides common hardware and high-level libraries like `hardware_gpio` and `pico_time`, it also pulls in system components like `pico-runtime`, which is needed to set up the hardware and runtime environment that lets you just implement `main()` and `pico_standard_link` which configures the linking of your executable whilst using a simple `CMakeLists.txt`. These are incredibly low-level components that most users will not need to worry about. The reason they are mentioned is to point out that they are ultimately *implicit dependencies* of your program because of your dependence on `pico_stdlib`; if you choose not depend on `pico_stdlib` and then you can pick just the exact SDK libraries you want explicitly.